

- 人工智能实验报告 第1周
 - 任务一：
 - 一.实验题目：二分查找
 - 二.实验内容
 - 1.算法原理
 - 2.关键代码展示(可选)
 - 三.实验结果及分析
 - 1.实验结果展示示例（可截图可表可文字，尽量可视化）
 - 2.评测指标展示及分析（机器学习实验必须有此项，其它可分析运行时间等）
 - 任务二：
 - 一.实验题目：矩阵加法，乘法
 - 二.实验内容
 - 1.算法原理
 - 2.关键代码展示(可选)
 - 三.实验结果及分析
 - 1.实验结果展示示例（可截图可表可文字，尽量可视化）
 - 2.评测指标展示及分析（机器学习实验必须有此项，其它可分析运行时间等）
 - 任务三：
 - 一.实验题目：字典遍历
 - 二.实验内容
 - 1.算法原理
 - 2.关键代码展示(可选)
 - 三.实验结果及分析
 - 1.实验结果展示示例（可截图可表可文字，尽量可视化）
 - 2.评测指标展示及分析（机器学习实验必须有此项，其它可分析运行时间等）

人工智能实验报告 第1周

姓名:陈安康 学号:22320021

任务一：

一.实验题目：二分查找

二.实验内容

1.算法原理

二分查找（Binary Search）是一种在有序数组中查找特定元素的高效算法。它通过反复将有序数组分成两半，并检查目标元素是否位于数组的中间，从而迅速缩小搜索范围。

假设查找下标范围是 $[low, high]$ ，设中间下标为 mid ，其中：

$$mid = (low + high) / 2$$

则判断下标 mid 处的数组值与目标值之间的关系，如果目标值大于数组值则从范围 $[mid+1, high]$ 范围内查找，小于则从 $[low, mid-1]$ 范围内查找，若相等则返回当前 mid 下标。搜索以两种方式结束：找到目标值并返回下标，或者 low 大于 $high$ 时，即整个数组内没有符合条件的目标值，返回-1。

接下来简略证明该算法的时间复杂度：

设有序数组长度为 n ，则每次查找近似将查找范围缩小为原来的一半，为 $n/2$ 。考虑最坏情况，即将搜索范围长度缩小为1时才能找出目标值或者退出搜索。设需要 i 次查找找出目标值或者退出查找，则有：

$$n/2^i = 1$$

可知 i 为 $\log(n)$ 的，所以此搜索算法时间复杂度为 $O(\log(n))$ 。

2.关键代码展示(可选)

```
def BinarySearch(nums, target):
    low=0
    high=len(nums)-1
    while low<=high:
        mid=int((low+high)/2)
        if nums[mid]<target:
            low=mid+1;
        elif nums[mid]>target:
            high=mid-1;
        else:
            return mid;
    return -1
```

三.实验结果及分析

1.实验结果展示示例（可截图可表可文字，尽量可视化）

以下是事例，给出一个升序数组，查找目标值6，因为目标值不在数组中，因此预期返回结果为-1，展示如下图所示：

```
12     ...return -1
13
14     test=[1,2,3,4,5,7,8,9,10]
15     index=BinarySearch(test,6)
16     print(index);
17
18
```

问题 输出 调试控制台 终端 端口

安装最新的 PowerShell，了解新功能和改进！ <https://aka.ms/PSWindows>

PS C:\Users\86135\Desktop\python> & 'c:\ProgramData\anaconda3\python.exe' 'c:\Users\86135\.vscode\extensions\ms-python.debugpy-2024.2.0-win32-x64\bundled\libs\debugpy\adapter\..\..\debugpy\launcher' '63118' '--' 'C:\Users\86135\Desktop\python\task1.py'

-1

PS C:\Users\86135\Desktop\python>

下图展示了目标值在数组中时的结果：

```
13
14 test=[1,2,3,4,5,7,8,9,10]
15 index=BinarySearch(test,3)
16 print(index);
17
18
```

问题 输出 调试控制台 终端 端口

```
cher' '63343' '--' 'C:\Users\86135\Desktop\python\task1.py'
```

```
2
```

```
PS C:\Users\86135\Desktop\python> c:; cd 'c:\Users\86135\Desktop\python'; & '
c:\ProgramData\anaconda3\python.exe' 'c:\Users\86135\.vscode\extensions\ms-pyt
hon.debugpy-2024.2.0-win32-x64\bundled\libs\debugpy\adapter\..\..\debugpy\laun
cher' '63353' '--' 'C:\Users\86135\Desktop\python\task1.py'
```

```
2
```

```
PS C:\Users\86135\Desktop\python>
```

(python)

行 15, 列 26 空格: 4 UTF-8 CRLF Python 3.11

2.评测指标展示及分析（机器学习实验必须有此项，其它可分析运行时间等）

在算法原理部分已经分析了其运行时间复杂度为 $O(\log(n))$ 。在导入时间模块后，可知在最坏情况下程序运行时间：

```
task1.py ×
task1.py > index
1  import time
2  def BinarySearch(nums,target):
3      low=0
4      high=len(nums)-1
5      while low<=high:
6          mid=int((low+high)/2)
7          if nums[mid]<target:
8              low=mid+1;
9          elif nums[mid]>target:
10             high=mid-1;
11         else:
12             return mid;
13     return -1
14
15     start_time = time.time()
16     test=[1,2,3,4,5,7,8,9,10]
17     index=BinarySearch(test,10)
18     print(index);
19     end_time = time.time()
20     run_time = end_time - start_time
21     print(run_time)
22
23

问题  输出  调试控制台  终端  端口
0.0
PS C:\Users\86135\Desktop\python> c:; cd 'c:\Users\86135\Desktop\python'; & 'c:\ProgramData\anaconda3\python.exe' 'c:\Users\86135\.vscode\extensions\ms-python.debugpy-2024.2.0-win32-x64\bundled\libs\debugpy\adapter\..\..\debugpy\launcher' '63566' '--' 'C:\Users\86135\Desktop\python\task1.py'
8
0.0010144710540771484
PS C:\Users\86135\Desktop\python>
```

从上述运行时间来看，运行效果可以接受。

任务二：

一.实验题目： 矩阵加法，乘法

二.实验内容

1.算法原理

输入两个矩阵，按照矩阵加法与乘法规则进行计算。设大写字母表示矩阵，它们中的元素用对应小写字母表示。两个 $n \times n$ 的矩阵为 **A** 和 **B**，它们相加，相乘的结果 **C** 中的元素求解表达式分别为：

$$c[i][j] = a[i][j] + b[i][j]$$

$$c[i][j] = \sum_{k=0}^{n-1} a[i][k] * b[k][j]$$

2.关键代码展示(可选)

```
def MatrixAdd(A,B):
    n=len(A)
    ans=[]
    for i in range(n):
        row=[]
        for j in range(n):
            row.append(A[i][j]+B[i][j])
        ans.append(row)
    return ans

def MatrixMul(A,B):
    n=len(A)
    ans=[]
    for i in range(n):
        row=[]
        for j in range(n):
            tmp=0
            for k in range(n):
                tmp+=A[i][k]*B[k][j]
            row.append(tmp)
        ans.append(row)
    return ans
```

三.实验结果及分析

1.实验结果展示示例（可截图可表可文字，尽量可视化）

展示结果如下：

```

23
24
25     matrix1 = [[1, 2, 3],
26                 [4, 5, 6],
27                 [7, 8, 9]]
28
29     matrix2 = [[9, 8, 7],
30                 [6, 5, 4],
31                 [3, 2, 1]]
32
33     ans1=MatrixAdd(matrix1,matrix2)
34     ans2=MatrixMul(matrix1,matrix2)
35
36     for cell in ans1:
37         print(cell)
38     print()
39     for cell in ans2:
40         print(cell)

```

问题 输出 调试控制台 终端 端口

op\python\task2.py'

```

[10, 10, 10]
[10, 10, 10]
[10, 10, 10]

```

```

[30, 24, 18]
[84, 69, 54]
[138, 114, 90]

```

2.评测指标展示及分析（机器学习实验必须有此项，其它可分析运行时间等）

该程序采用对数学计算方法过程的模拟。对时间复杂度进行分析，设输入规模为 $n*n$ 矩阵，则由于循环占主要操作，加法函数复杂度为 $O(n^2)$,乘法函数的时间复杂度为 $O(n^3)$

。

任务三：

一.实验题目：字典遍历

二.实验内容

1.算法原理

对每一个字典键值对进行修改，使其互换。

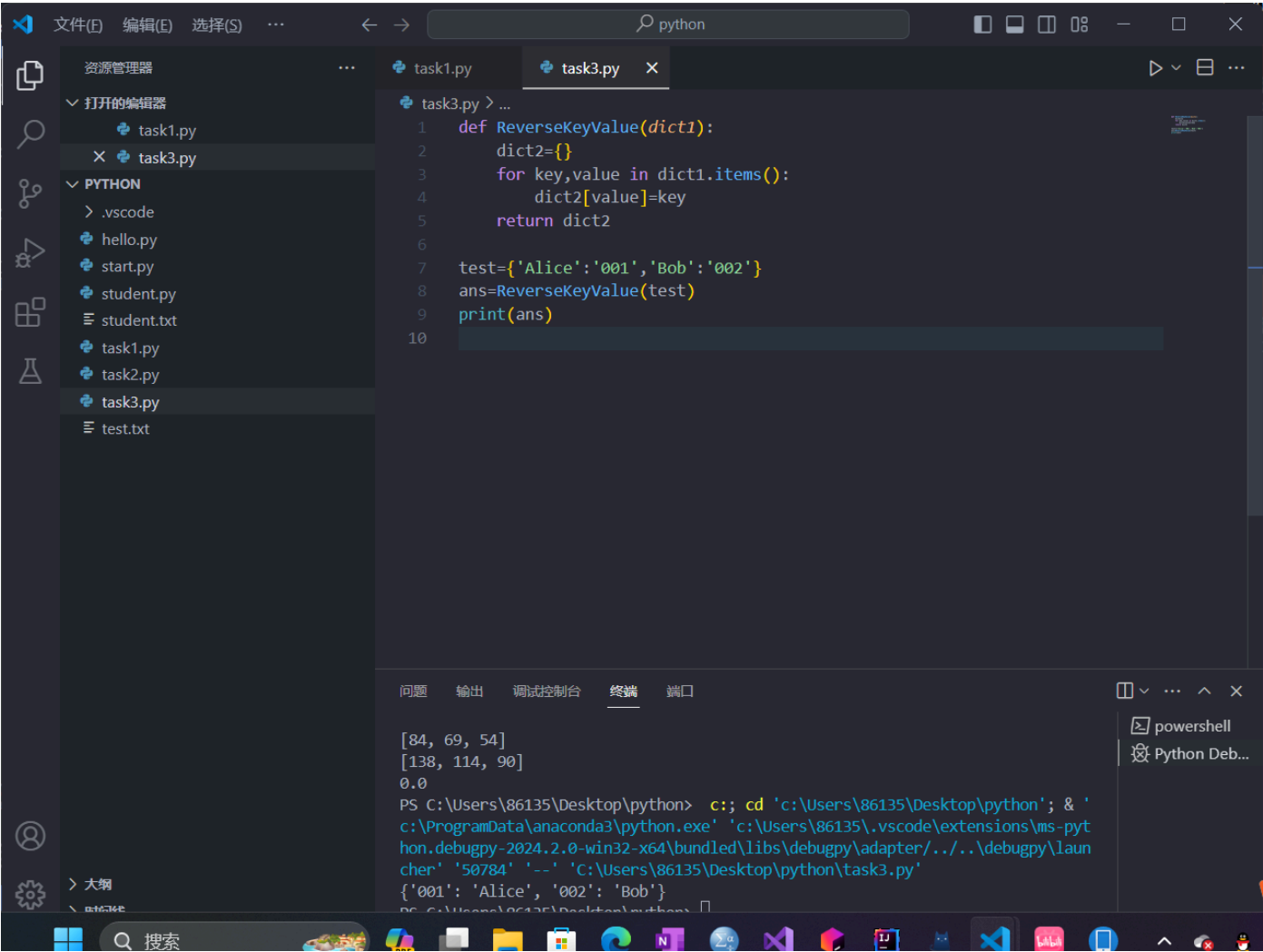
2.关键代码展示(可选)

```
def ReverseKeyValue(dict1):
    dict2={}
    for key,value in dict1.items():
        dict2[value]=key
    return dict2
```

三.实验结果及分析

1.实验结果展示示例（可截图可表可文字，尽量可视化)

展示结果如下：



2.评测指标展示及分析（机器学习实验必须有此项，其它可分析运行时间等）

易知时间复杂度为 $O(n)$ 。